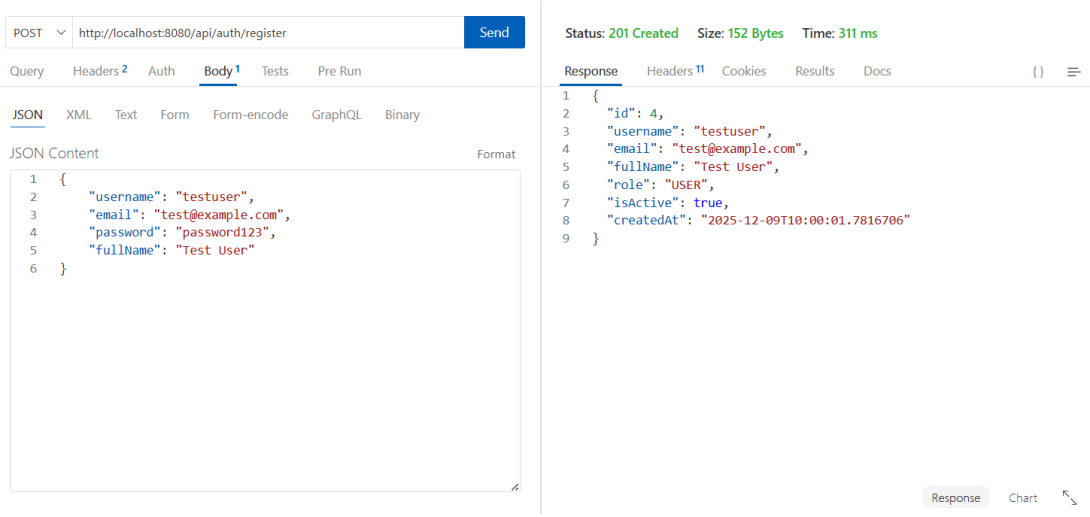
WEB APPLICATION DEVELOPMENT
LAB 9 - INCLASS

I. Test Registration



Explanation of the code flow of registeration

1. **Client → Server: Send Registration Request.**

– The frontend (Thunder Client/Postman) calls: POST /api/auth/register including username, email, password, fullName.

2. Controller layer handles the request: The controller (AuthController) receives the JSON and maps it to a DTO object: public ResponseEntity<?> register(@RequestBody RegisterRequest request)

– Controller responsibilities for validating input, call AuthService.register(request).

3. Service layer (AuthService) runs business logic

– Server → Check for existing username or email

– Server → Hash password with Bcrypt

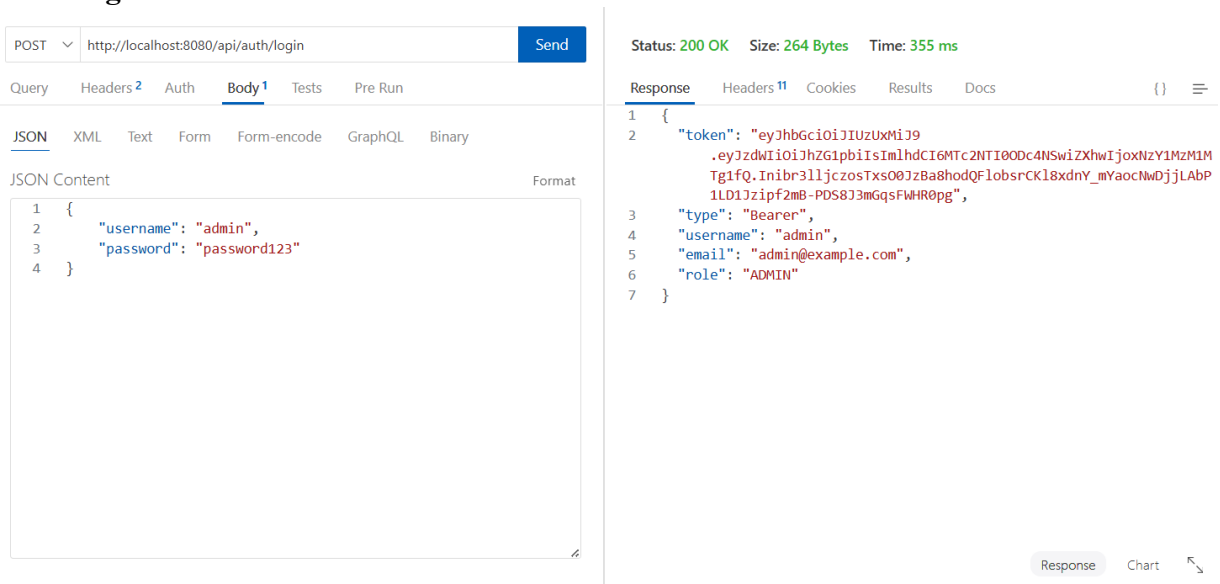
– Server → Create user entity

– Server → Save user to database

4. Repository layer persists data: Spring Data JPA performs:
INSERT INTO users (username, email, password, full_name, role, is_active)
VALUES (...)

5. Server → Client: Send Response (201 Created)

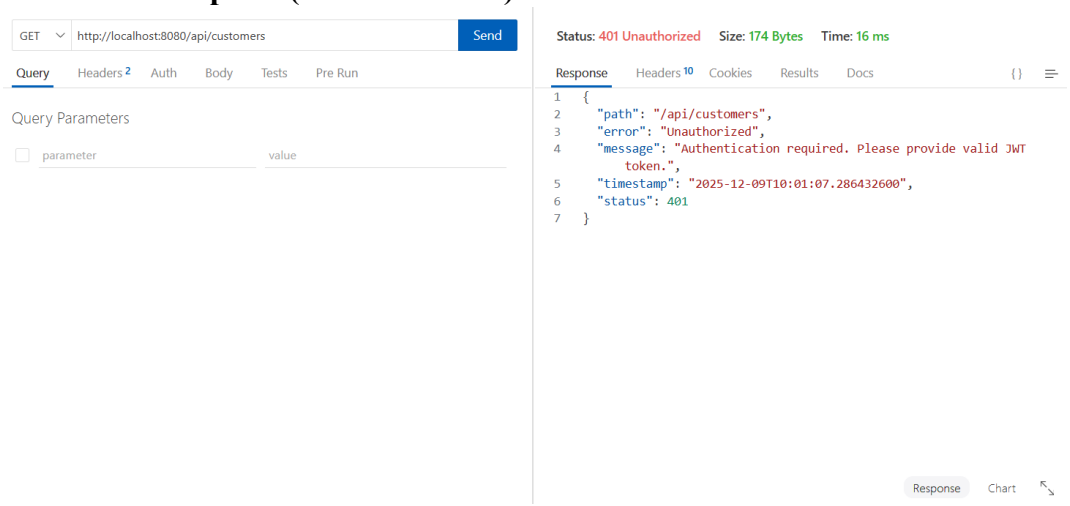
II. Test Login



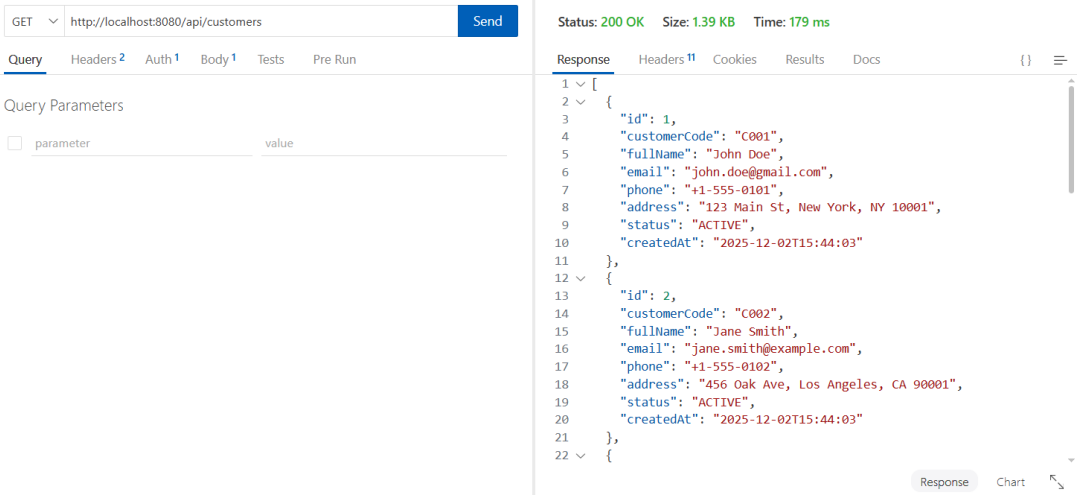
Explanation of the code flow of the login

1. Client → POST /api/auth/login {username, password}. The frontend or API client sends login credentials to the backend. The purpose of this is authenticating the user, obtaining a JWT token for future requests.
2. Controller receives the request: AuthController maps the JSON to a LoginRequest DTO: public ResponseEntity<?> login(@RequestBody LoginRequest request).
3. Server → Verify Username Exists.
4. Server → Compare Password with Hashed Password: Spring Security performs this comparison using BCryptPasswordEncoder.matches().
5. Server → Generate JWT token with username, role, issued time, expiry
6. Server → Return: {token: "eyJhbG...", username, email, role}
7. Client → Store token in localStorage

III. Test Protected Endpoint (Without Token)



IV. Test Protected Endpoint (With Token)



Explanation of the code flow of the accessing protected endpoint:

1. Client → GET /api/customers
   Header: Authorization: Bearer eyJhbG...

– The frontend or API client sends a request to a protected resource.
– Tell the server that "I am an authenticated user; here is my token."

2. JwtAuthenticationFilter intercepts request

– Spring Security applies to your custom filter before reaching the controller.

3. Server → Extract token from Authorization header

– The filter reads the HTTP header: String authHeader = request.getHeader("Authorization");

4. Server → Validate token (signature, expiry)

– JWT service performs checks:

+ Verify the token was signed with your secret key (jwt.secret)
+ Check expiration time (exp)
+ Check token integrity (tampering detection)
+ If validation fails → 401 Unauthorized.

5. Server → Extract username from token. JWT contains sub, role, exp.

– The filter extracts the subject: String username = jwtService.extractUsername(token);

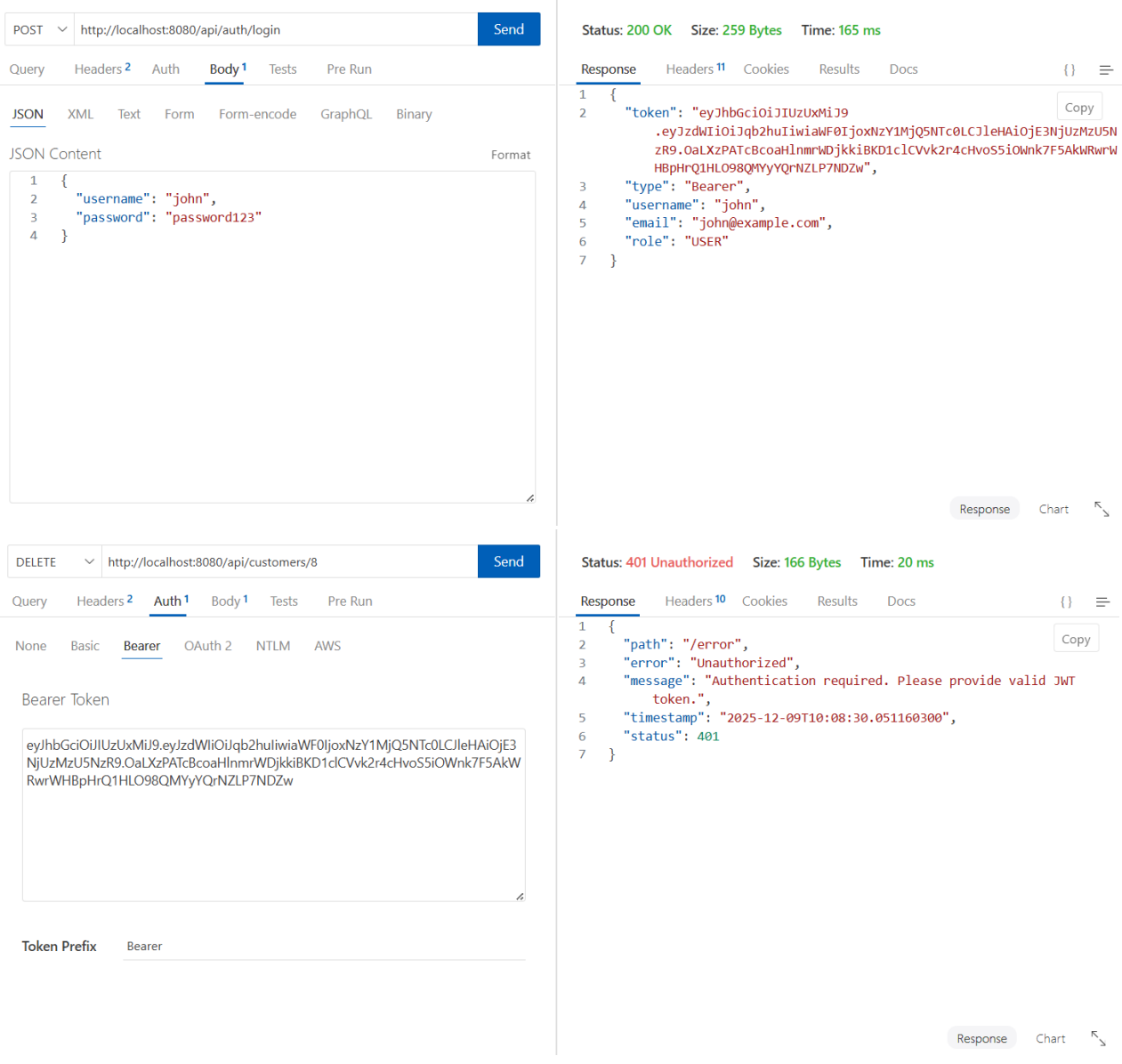
6. Server → Load user details from database: Spring loads the authenticated user: UserDetails userDetails = userDetailsService.loadUserByUsername(username);.
7. Server → Set authentication in SecurityContext
8. Server → Process request Server → Return response

– Now that the user is authenticated, Spring calls your controller method:
@GetMapping("/api/customers")
public List<Customer> getAllCustomers() { ... }

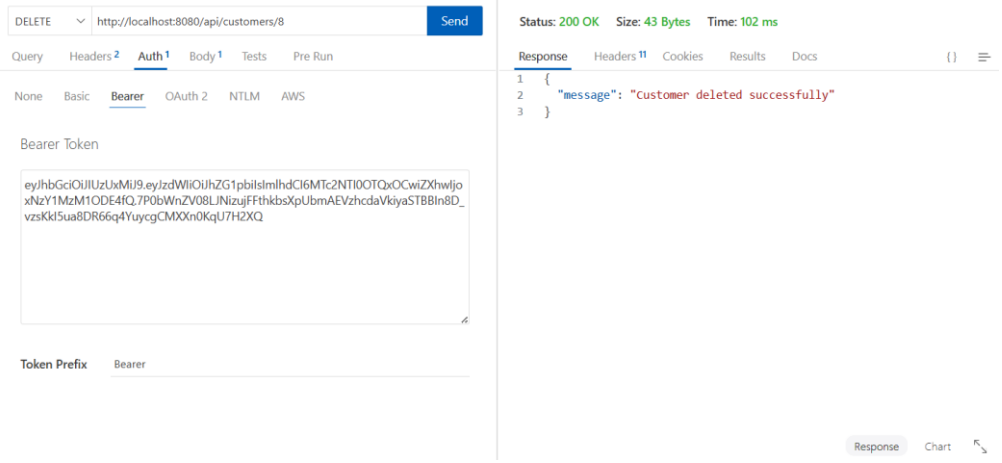
– The controller reads data, service executes logic, and repository queries database.

9. Server → Return response a valid JSON

V. Test Authorization (USER trying to DELETE)



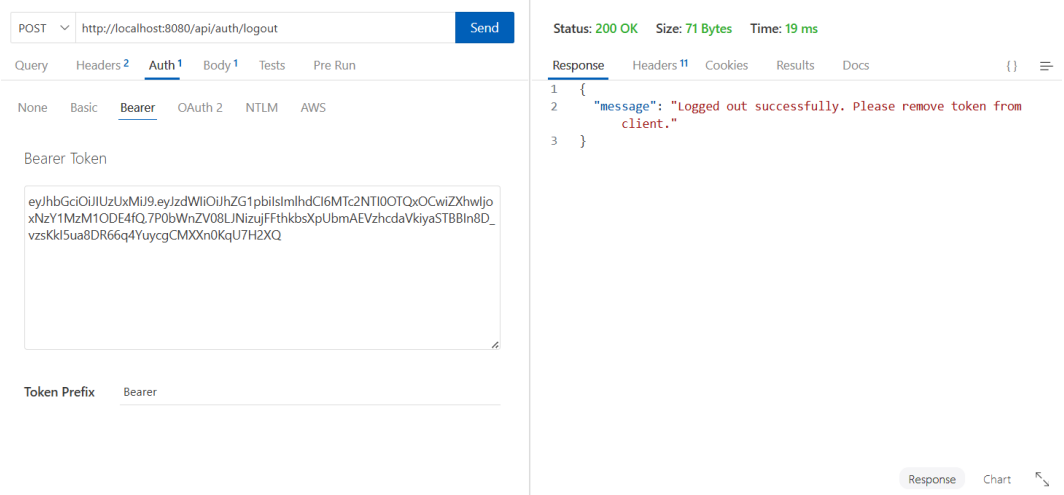
VI. Test Authorization (ADMIN can DELETE)



Explanation of the code flow of authorization that user/admin trying to delete

1. Client → Logs in as USER ("john")
– Client sends: POST /api/auth/login {"username": "john", "password": "password123"}
– Server returns a JWT: { "token": "eyJh…", "username": "john", "role": "USER"}
2. Client → Attempts DELETE /api/customers/8
– Client sends: DELETE /api/customers/8. Authorization: Bearer eyJh… This endpoint is protected and requires ADMIN authority.
3. JwtAuthenticationFilter intercepts the request: The flow inside the filter:
– Extract token
– Validate token
– Extract username (“john”)
– Load user from database
– Set authentication (role = USER)
4. Authorization Check Happens in SecurityFilterChain. Now Spring Security compares:
– Required role: **ADMIN**
– User's role: **USER**
– Role mismatch → Access denied
5. Server denies request → Returns 401 or 403. Depending on your configuration:
– If the JWT is missing or invalid → **401 Unauthorized**
– If the JWT is valid but role is insufficient → **403 Forbidden**
6. Error Response Returned. Server responds: {"path": "/error", "error": "Unauthorized", "message": "Authentication required. Please provide valid JWT token.", "status": 401}

VII. Logout (ADMIN logout)



Explanation of the code flow of logging out

1. Client → POST /api/auth/logout. This call typically does **not** include pasword, JWT token, user information because the server does not need them to invalidate anything.
2. Server → Handle logout request.
3. Client → Remove token from localStorage. This is the **real logout action**. The frontend removes: localStorage.removeItem("token");.
4. Server → Return: {message: "Logged out"}